VAMP, cuRobo and PyRoki on one Jetson Orin NX: planning speed, executable trajectories and industrial motion types

vamp-bench technical report

2026-10-15

1 Summary

This report compares motion planners on a single Jetson Orin NX 16 GB. VAMP is a vectorised sampling-based planner that runs on one CPU core. cuRobo v1 and cuRoboV2 are GPU trajectory optimisers. PyRoki, a JAX trajectory-optimisation library with no global planner, is reported in its own category. All planners check the same collision spheres against the same obstacles and are compared only on this board. The work has three parts.

- **v1** measures planning time on VAMP's standard benchmark and on the scenes of our earlier paper.
- **v2** measures the time to a trajectory that passes a common 1 kHz executability check, on a held-out set and in pick-place.
- **v2.2** tests motions that cell integrators program: waypoint paths with blends, circular moves (MoveC) and straight lines with a trapezoidal speed profile.

Each part was pre-registered before it ran. Changes after registration are logged as amendments.

Takeaways.

1. **Planning time.** On this board, VAMP on one CPU core plans faster than cuRobo on the whole GPU.
   - **v1:** cuRobo v1's median planning time is 169× to 483× VAMP's on the same queries, and cuRoboV2's is 684× to 3817×.
   - **v2:** counting time-parameterisation, VAMP reaches an executable trajectory 59× to 478× sooner than cuRobo v1, in every bin of every set.
2. **Cycle time is much closer than planning time**, because execution takes seconds.
   - **Held-out runs both planners solve:** cuRobo v1's cycle is shorter (2.36 s against 2.67 s). Its smoother trajectories execute 0.97 s faster, which outweighs VAMP's 0.21 s planning advantage.
   - **Counting failures:** VAMP's median cycle is lower (3.09 s against 3.76 s).
3. **The finding of v2.2 is expressibility.** Neither cuRobo version can follow a programmed Cartesian path: it holds a straight line only along an axis of the goal or base frame.
   - **Lines:** none of the 583 programmed lines runs along such an axis.
   - **Other motions:** waypoint paths run as separate lines that stop at every via point. MoveC and a programmed speed profile cannot be expressed.
   - **PyRoki:** it expresses all three motion types, and follows the trapezoidal line closely. As returned, its blended and circular trajectories mostly exceed the joint jerk limit.

2 Setup

- **Hardware:** one Jetson Orin NX 16 GB.
  - VAMP runs on one Cortex-A78AE core with NEON. cuRobo and PyRoki run on the GPU, with no other GPU work during their runs.
  - No result here is transferred from other hardware.
- **Planners:**
  - VAMP `27cb9b6` with its library defaults (RRT-Connect, simplification, Halton sampler);
  - cuRobo v1 `ebb7170` (v0.7.7-5), the version our earlier paper used, with the settings of its own benchmark script;
  - cuRoboV2 `ac5f931` (v0.8.0.post1.dev33), with NVIDIA's benchmark settings except `max_attempts` 20 instead of 100;
  - PyRoki with its own example settings (Section 6).
- **Shared collision model:** all planners use the same sphere model of each robot, the same obstacles and the same joint-space start and goal.
  - On 16,000 random configurations, VAMP's and cuRobo's collision checkers agree on 100% of them.
- **Executability check (v2 onward):** every trajectory, from every planner, is sampled at 1 kHz and must pass all of the following:
  - no collision under the shared model;
  - joint bounds respected;
  - peak velocity, acceleration and jerk each at most 1.01 times the shared limits.

  The UR5e jerk limit is 500 $\mathrm{rad/s^3}$, taken from cuRobo's shipped configuration, since Universal Robots publishes none. Planning plus time-parameterisation (T_{ready}) must finish within T = 30 s.
- **Two versions of each trajectory:**
  - **R1** is the trajectory as returned.
  - **R2** applies a uniform slow-down to a trajectory that failed only on velocity, acceleration or jerk. The factor is the smallest that brings all three peaks within the limits (velocity scales with the factor, acceleration with its square, jerk with its cube). R2 is post-processing and is labelled wherever it appears.

3 v1: planning speed

The problem sets are VAMP's MotionBenchMaker problems (Panda, UR5) and the three scenes of our earlier paper (a UR5e on a rail, with and without pillars, and a fixed-base UR5e).

- **Timing:** wall-clock time per query, measured from Python, including the world update.
- **Solved:** the planner reports success.
- **cuRoboV2 subset:** V2 runs a fixed subsample of the MotionBenchMaker problems and one trial per paper problem, because of its run time.

Table 1: Solved queries and wall time per query (median, p95 in parentheses). Rail: the UR5e on a rail of our earlier paper; UR5e: its fixed-base UR5e scene.

	VAMP, one CPU core		cuRobo v1, GPU		cuRoboV2, GPU	
Panda MBM	699/699	1.76 ms (3.89)	638/699	849 ms (2.69 s)	59/69	6.55 s (13.9)
UR5 MBM	689/689	2.02 ms (4.80)	367/689	827 ms (2.18 s)	35/64	4.52 s (16.6)
Rail, pillars	1020/1030	0.95 ms (1.35)	823/1030	161 ms (433)	100/103	1.06 s (2.28)
Rail, no obstacles	1030/1030	0.55 ms (0.68)	1030/1030	145 ms (154)	103/103	379 ms (727)
UR5e, pillars	1030/1030	0.97 ms (1.43)	985/1030	377 ms (613)	102/103	1.05 s (2.32)

Table 2: Median wall time relative to VAMP on the same queries.

	Panda MBM	UR5 MBM	Rail, pillars	Rail, no obstacles	UR5e, pillars
cuRobo v1	483×	409×	169×	261×	388×
cuRoboV2	3817×	2225×	1117×	684×	1079×

Panda MotionBenchMaker.

- **Medians:** VAMP 1.76 ms, cuRobo v1 849 ms, cuRoboV2 6.55 s.
- **cuRoboV2's success is a lower bound:** it solves 59/69 with `max_attempts` 20. NVIDIA's own benchmark uses 100.
- **v1 on the same subset:** it solves 64/69 with a median of 869 ms.

On this board the GPU gives cuRobo no planning-time advantage, and cuRoboV2 is slower than v1.

Notes.

- **UR5 velocity limit:** VAMP's UR5 model caps every joint at a velocity cuRobo's time-parameterised optimiser cannot work with. Both cuRobo versions therefore receive the real UR5 limit, with identical geometry. cuRobo v1 still solves only 367/689.
- **Provisional, penetrations between waypoints:** VAMP's straight-line check between consecutive returned waypoints flags a few successes of both cuRobo versions (Table 3).
  - **Size:** median penetration 0.12–2.30 mm per set, at most 5.08 mm, never at a waypoint.
  - **Open question:** whether the continuous cuRobo trajectory itself penetrates is not established.
  - The solved counts above are not changed.

Table 3: Provisional: successes flagged by VAMP's segment check between returned waypoints.

	cuRobo v1	cuRoboV2
Panda MBM	0 of 638	4 of 59
UR5 MBM	4 of 367	6 of 35
Rail, pillars	44 of 823	16 of 100
Rail, no obstacles	0 of 1030	0 of 103
UR5e, pillars	3 of 985	4 of 102

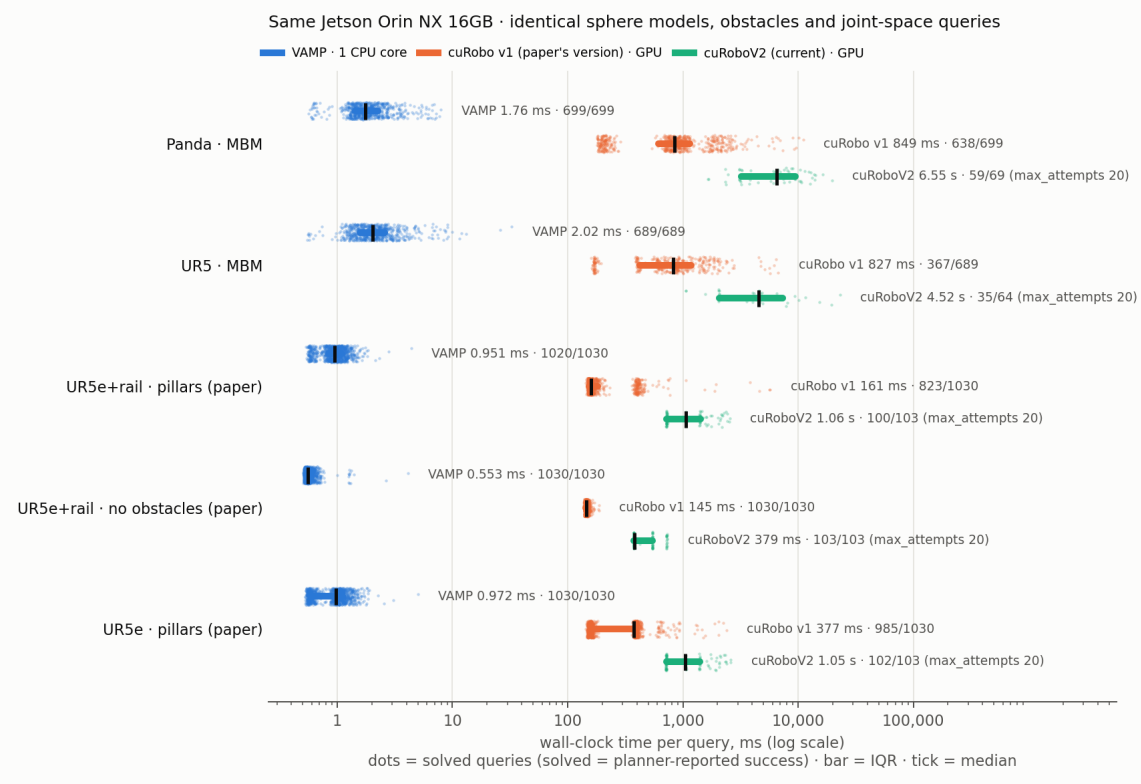


Figure 1: Wall-clock time per solved query on each problem set (log scale). Bars show the interquartile range, ticks the median.

4 v2: time to an executable trajectory

v1 measures time to a path. v2 measures time to a trajectory a controller can execute.

- **What counts:** a planner succeeds only if its output passes the executability check within T = 30 s. VAMP's paths are time-parameterised with Ruckig, and the Ruckig time counts.
- **Sets:**
  - **Held-out cells (primary):** 300 UR5e-on-rail queries generated after a frozen commit. Each query is a pair of collision-free configurations at two stations of a cell, not a tool-down pick.
  - MotionBenchMaker and MπNets Panda queries.
- **Seeds:** 10 per query. cuRoboV2 runs a 100-query subsample with 3 seeds.
- **Feasibility:** a reference planner certifies 299 of 300 held-out queries, 300 of 300 MotionBenchMaker queries and 299 of 300 MπNets queries as feasible.

Table 4: Held-out cells (UR5e on a rail). cuRoboV2 on its subsample; on the same runs VAMP succeeds on 95.0% and cuRobo v1 on 64.7%.

	VAMP	cuRobo v1	cuRoboV2
Success (runs)	93.1%	56.9%	71.0%
Queries failing on some seed / every seed	100 / 3	160 / 95	29 / 29
Median T_{ready}, solved runs	3.54 ms	210 ms	1.51 s
Median T_{exec}, solved runs	3.03 s	2.06 s	2.16 s
Median T_{cycle} (failures at T)	3.09 s	3.76 s	4.06 s

Table 5: Pre-registered hypotheses H1–H4, per set.

	Held-out	MotionBenchMaker	MπNets
H1: VAMP's T_{ready} lower in every bin	yes	yes	yes
T_{ready} ratio, cuRobo v1 / VAMP	59×	478×	159×
H2: cycle-time gap smaller (T_{cycle} ratio)	yes (1.22)	yes (1.14)	yes (0.99)
T_{cycle}, runs both solve, VAMP / v1	2.67 / 2.36 s	2.53 / 3.10 s	1.65 / 1.81 s
H3: v1 lower jerk / shorter T_{exec}	yes / yes	yes / yes	yes / yes
VAMP + TOPP-RA: T_{exec}, share valid	1.79 s, 48%	1.55 s, 56%	1.14 s, 71%
H4: queries failing on some seed, VAMP / v1	100 / 160	231 / 55	128 / 17
H4: more robust	VAMP	cuRobo v1	cuRobo v1

Results.

- **Time to an executable trajectory:** VAMP reaches one 59× (held-out), 478× (MotionBenchMaker) and 159× (MπNets) sooner than cuRobo v1 in median. This holds in every bin of every set (Figure 2).
- **Cycle time:** the gap is far smaller (Table 5, H2).
  - **Held-out runs both planners solve:** cuRobo v1's cycle is shorter, because its execution is shorter.
  - **Counting failures at T:** VAMP's median cycle is lower.
- **Smoothness:** cuRobo v1's trajectories have lower jerk and shorter execution than VAMP with Ruckig, on every set (H3).
  - **VAMP with TOPP-RA:** it executes faster, but only 48% to 71% of its trajectories pass the check.
- **Robustness differs by set (H4):** VAMP has fewer queries failing on some seed on the held-out set, and cuRobo v1 has fewer on MotionBenchMaker and MπNets.
- **cuRoboV2:** its median T_{ready} is 7× to 15× cuRobo v1's on the same queries and seeds.
- **Appendix set (H5):** on the held-out set with its original, unwrapped goals, 156 of 300 queries are certified. The signs of H1, H2, H3 (jerk), H3 (execution time) agree with the primary set; H4 does not.

4.1 Pick-place with straight-line approach and retreat

- **Task:** 30 cells with 4 picks each, run with 5 seeds, for 600 picks per planner.
- **Picks:** each pick has 6 segments: free-space moves to the pre-grasp and pre-place poses, and straight-line (MoveL) moves down and up at each.
- **Success:** a pick counts only if every segment passes the check.

Results.

- **Picks completed:** VAMP 587/600, cuRoboV2 585/600, cuRobo v1 as configured 0/600.
- **cuRobo v1 as configured:** its MoveL trajectories exceed its own configured jerk limit at the segment ends (Figure 3). Only 200/2304 of its MoveL segments pass.
  - **Lower jerk setting:** with `max_jerk` lowered to 200, it completes 572/600.

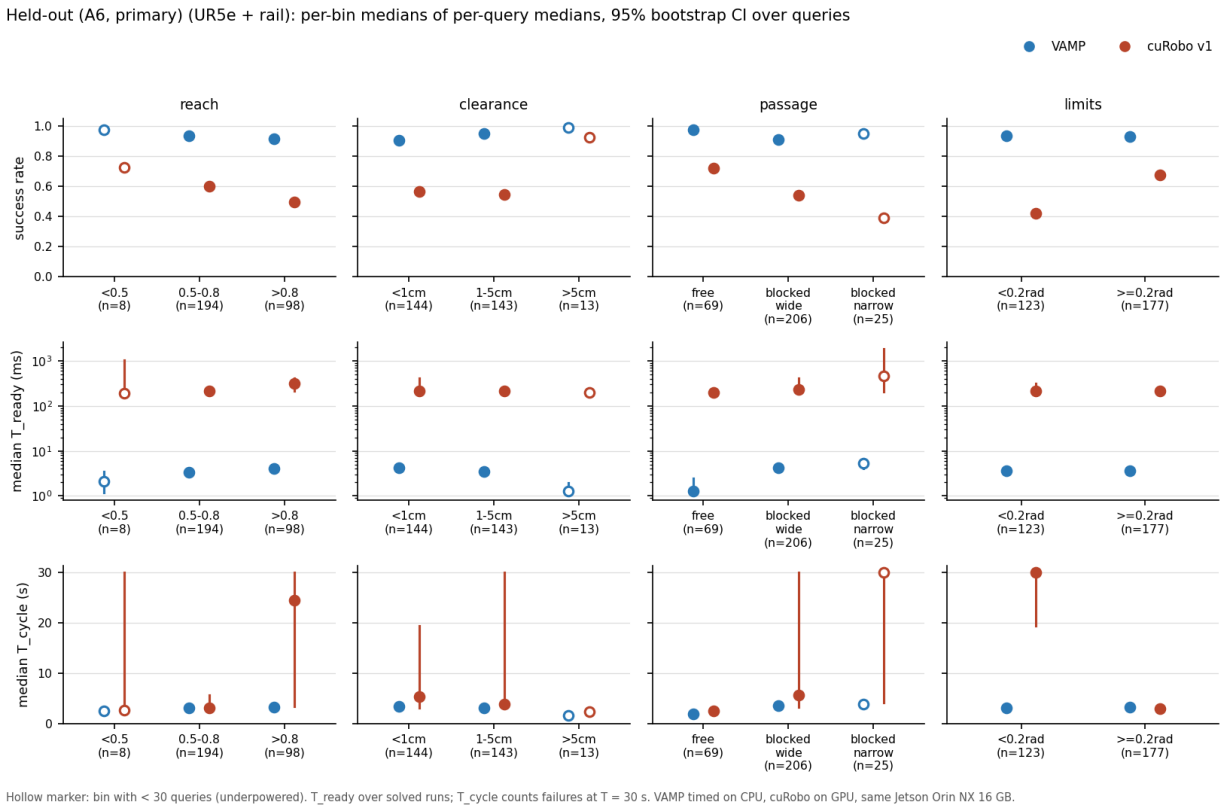


Figure 2: Held-out cells: success rate, T_{ready} and T_{cycle} per bin, with bootstrap confidence intervals over queries. Hollow markers are bins with fewer than 30 queries.

  - **Uniform slow-down (R2, post hoc):** with a median factor of 1.27 it completes 573/600 at a median cycle of 8.15 s.

- **Paired cycle times:**
  - **VAMP and cuRoboV2** (576 picks both complete): 6.07 s against 18.88 s, a median ratio of 0.32.
  - **VAMP and the v1 control** (566 picks): 6.06 s against 8.22 s, a ratio of 0.74.

  Execution times are close, so the differences are planning time.

- **Timing caveat:** VAMP's pick-place planning ran while other jobs loaded the CPU, so its planning times are upper bounds.

5 v2.2: industrial motion types

Programs. 100 UR5e cells each carry three programmed tool-down motions:

- **W:** a path through 3–6 via points with circular blends (radius up to 5 cm), at a constant 0.25 m/s through the blends;
- **C:** a horizontal circular arc through start, via and end points (MoveC);
- **T:** a straight line of 0.30–0.59 m with a trapezoidal TCP speed profile (acceleration 0.25 m/s^2, cruise 0.25 m/s).

Each time law is jerk-limited along the path (TCP jerk 10 m/s^3).

Success. A run must pass all of the following:

Table 6: Pick-place with MoveL (UR5e on a rail).

	VAMP	v1 as configured	v1, max_jerk 200	cuRoboV2
Picks completed	587/600	0/600	572/600	585/600
Pick slots failing on some seed	8/120	120/120	11/120	3/120
Median cycle per completed pick	6.08 s	–	8.22 s	18.92 s
planning	0.35 s	–	2.76 s	13.71 s
execution	5.72 s	–	5.47 s	5.34 s
MoveL TCP deviation, median / p95 (mm)	0.04 / 0.07	0.04 / 0.49	0.03 / 0.34	0.65 / 0.92
MoveL orientation deviation, p95	0.03°	1.89°	1.51°	0.05°

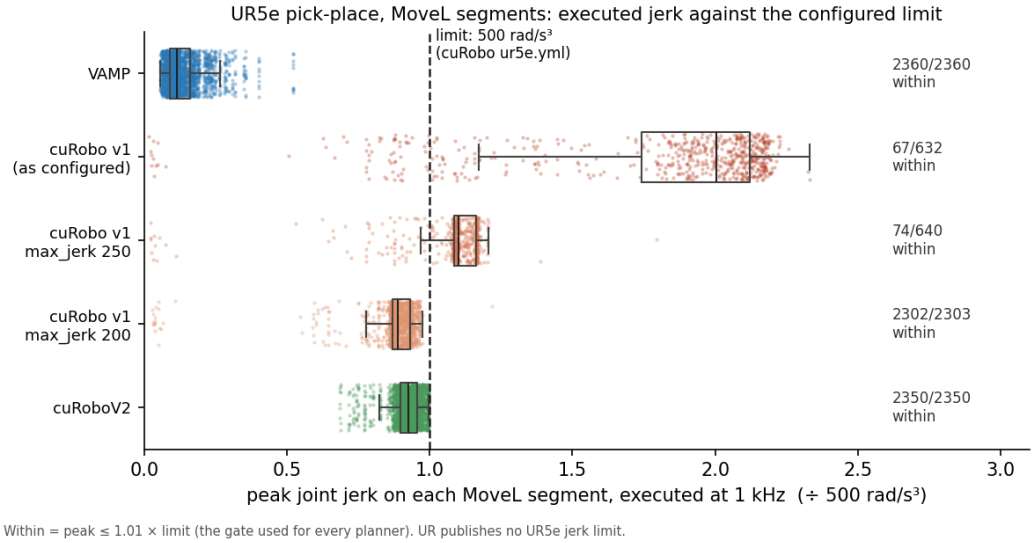


Figure 3: Peak joint jerk on each executed MoveL segment, relative to the configured limit.

- the executability check;
- planning within T;
- an end pose within 1 mm and 1°;
- a TCP within 5 mm of the programmed path outside blend zones, with orientation within 1°.

Speed, blend and profile behaviour are measured but are not part of success, because some planners have no speed input.

Runs. cuRobo v1 ran with 10 seeds per program, both as configured and with `max_jerk` lowered as in pick-place. cuRoboV2 ran with 3 seeds, and PyRoki once (it is deterministic).

Cartesian pipeline. This column is our own path IK, spline and time law, with VAMP used only as the collision check. It is a reference for what the programmed motion costs on this robot, not a result about VAMP's planning.

5.1 Expressibility

- **How cuRobo holds a line:** both versions hold a straight line by holding axes of the goal frame or the base frame. An axis can be held only when the line's displacement along it is at most 1 mm.
  - A tool-down line in an arbitrary direction usually has no such axis. Only orientation is then held, and the TCP path between the endpoints is unconstrained.

Table 7: How each planner expresses each motion type. This is the main v2.2 result.

Motion	cuRobo v1 and cuRoboV2	PyRoki	Cartesian pipeline
W, waypoints with blends	Closest native motion: one line per leg, stopping at each via point. Of 483 legs, 0 run along a holdable axis, 15 can hold one axis, and 468 hold orientation only.	Per-knot pose costs (our harness)	Our post-processing
C, MoveC	Not expressible; not run	Per-knot pose costs	Our post-processing
T, trapezoidal line	Line only, by the same rule: of 100 lines, 0 along an axis, 1 plane, 99 orientation only. The speed profile is not expressible (no TCP speed or acceleration input).	Per-knot targets at the programmed timing	Our post-processing; profile exact

- **cuRoboV2's per-timestep targets:** its interface documents per-timestep tool-pose targets, but its trajectory optimiser rejects a goal of more than one timestep. A probe on development cells, logged before the measured runs, confirmed this.
- **What the cuRobo results measure:** cuRobo runs these programs through its closest native motion. Its results below measure that gap, not its free-space planning.

5.2 Results per motion type

Table 8: W, waypoint paths with blends (100 programs).

	Cartesian pipeline	cuRobo v1	v1, max_jerk 200	cuRoboV2	PyRoki
Success, as returned (R1)	100/100	0/1000	10/1000	3/300	1/100
Success after slow-down (R2)	100/100	10/1000	10/1000	3/300	19/100
Programs failing on every seed	0	100	99	99	99
Max path deviation, all runs, median / p95 (mm)	0.03 / 0.04	24.35 / 154.26	24.36 / 155.74	23.42 / 457.93	1.82 / 3.78
Cruise speed / programmed, median	0.50	1.98	1.69	1.54	0.99
Via points: median min speed / programmed	0.50	0.00	0.00	0.00	0.95
Via points stopped / held	0% / 22%	100% / 0%	100% / 0%	100% / 0%	0% / 73%
Median planning time, all runs	0.62 s	2.63 s	2.64 s	4.21 s	41.00 s

cuRobo.

- **Waypoint paths:** one program succeeds (3/300 for cuRoboV2, 10/1000 for v1 with the lower jerk setting). Its legs all hold orientation only and happen to stay within the path limit. Every cuRobo waypoint path stops at the via points.
- **T lines:** cuRoboV2 succeeds on 120/300 runs, v1 with the lower jerk setting on 319/1000 and v1 as configured on 29/1000.
  - **Path:** over all runs the median maximum deviation is 7.6–9.3 mm. On successful runs it is 2.3–3.6 mm, so the pre-registered path-fidelity mark (median at most 1 mm) is not met.

Table 9: C, circular arcs (100 programs).

	Cartesian pipeline	cuRobo v1	v1, max_jerk 200	cuRoboV2	PyRoki
Success, as returned (R1)	100/100	not expressible	not expressible	not expressible	3/100
Success after slow-down (R2)	100/100	–	–	–	88/100
Programs failing on every seed	0	–	–	–	97
Max path deviation, all runs, median / p95 (mm)	0.03 / 0.04	–	–	–	1.21 / 4.72
Cruise speed / programmed, median	1.00	–	–	–	1.00
Median planning time, all runs	0.20 s	–	–	–	11.72 s

Table 10: T, straight lines with a trapezoidal profile (100 programs).

	Cartesian pipeline	cuRobo v1	v1, max_jerk 200	cuRoboV2	PyRoki
Success, as returned (R1)	100/100	29/1000	319/1000	120/300	84/100
Success after slow-down (R2)	100/100	328/1000	319/1000	120/300	99/100
Programs failing on every seed	0	91	66	60	16
Max path deviation, all runs, median / p95 (mm)	0.03 / 0.04	9.27 / 39.20	9.01 / 39.13	7.59 / 28.10	0.21 / 1.08
Cruise speed / programmed, median	1.00	4.93	4.63	4.36	1.00
Ramp acceleration error, median	2%	2646%	2257%	2083%	7%
Ramp duration / programmed, median	1.00	0.18	0.20	0.23	1.00
Median T_{cycle}, successful runs	2.90 s	1.22 s	1.14 s	1.57 s	11.64 s
Median planning time, all runs	0.25 s	0.61 s	0.63 s	1.02 s	9.02 s

  - **Speed:** cuRobo moves the line at its own time-optimal speed, 4.4 to 4.9 times the programmed speed in median. Its ramps last 0.18–0.23 of the programmed duration.
  - **Cycle time:** its cycle is shorter than the Cartesian pipeline's on the programs both solve (cuRoboV2 1.57 s against 2.76 s over 40 programs). That cycle is not the cycle of the programmed motion.
  - **Line length:** success falls with length. cuRoboV2 succeeds on 52% of runs on lines shorter than the median and 28% on longer ones; for v1 with the lower jerk setting the figures are 46% and 18%.

- **cuRobo v1 as configured:** as in pick-place, its trajectories exceed the jerk limit (898 of 971 failed T runs).

PyRoki.

- **T lines:** it succeeds on 84/100, with a median deviation of 0.21 mm and a ramp acceleration error of 6.8%.
  - **Cycle time:** planning dominates (8.90 s planning, 2.68 s execution).
- **W and C, as returned:** the trajectories keep the programmed speed, including through

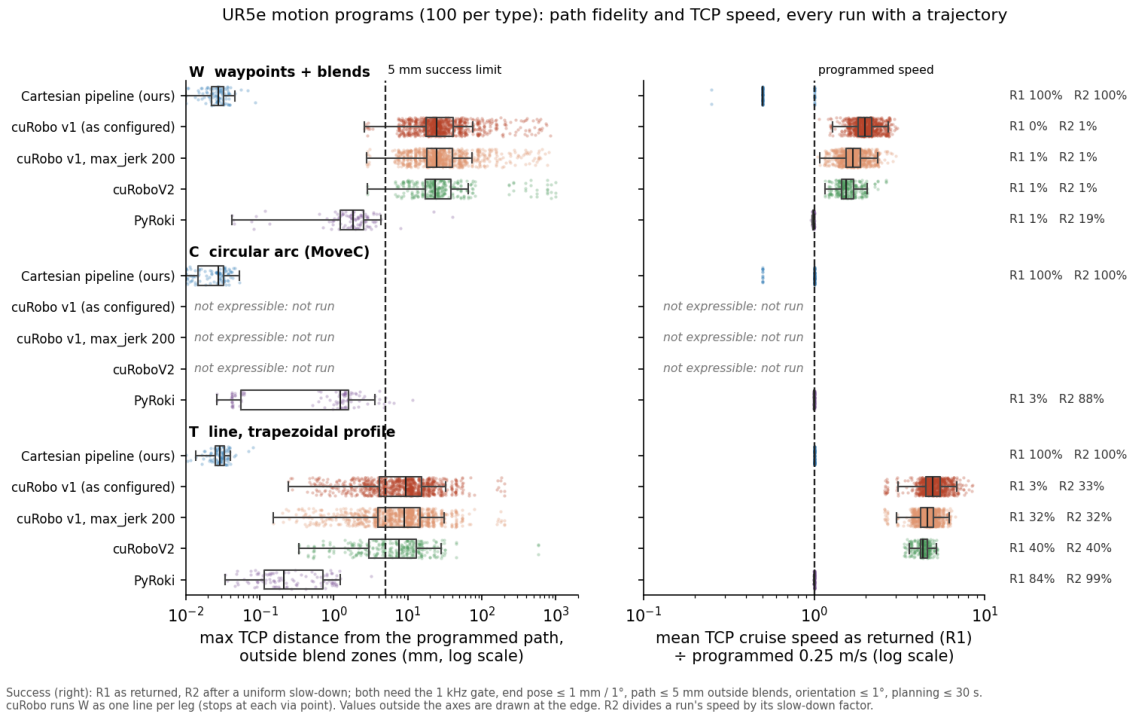


Figure 4: Path deviation outside blend zones (left) and cruise speed relative to the programmed speed (right), for every run with a trajectory.

the blends (median via-point speed 0.95 of programmed). But 98 of 99 failed W runs and 97 of 97 failed C runs exceed the jerk limit.

- **W and C after slow-down (R2):** 19/100 (W) and 88/100 (C) succeed, at median slow-downs of 1.26 and 1.62, so below the programmed speed.
- **Planning time on W:** with knots at most 40 ms apart, W planning takes a median of 41.0 s, and 79 programs exceed T.

Cartesian pipeline.

- **Fidelity and planning time:** it completes 300/300 programs and holds every path to 0.04 mm (p95). Planning takes a median of 0.20–0.62 s per program type.
- **Speed:** at the programmed speed, the circular blends exceed the joint limits on 76 of 100 waypoint paths, and so do 15 arcs. By the pre-registered rule it then halves the program speed.
- **Cycle time on W:** its median is 7.57 s, including the halved speed.

6 v2.1: PyRoki as a trajectory optimiser (provisional)

Status: provisional. The v2.1 write-up is pending. These numbers come from the stored runs, summarised by a script, and may be revised.

Correction first: MoveL succeeds on 373/480 segments, not 6/480.

- **The setup:** PyRoki ran the 480 MoveL segments of the 120 pick slots of the pick-place study, one run each, on the GPU.

- **The pre-registered run:** it reached the end pose only through the per-knot pose cost. It ended within 1 mm on only 13/480 segments (median end error 2.6 mm) and succeeded on 6/480 after slow-down.
- **Where the shortfall came from:** our harness, not PyRoki. With PyRoki's own endpoint recipe (both endpoint configurations held over the end knots, with a small rest cost), 475/480 segments end within tolerance (median 0.09 mm) and 373/480 succeed after slow-down. This rerun is logged as a post-hoc amendment.
- **Remaining failures:**
  - 99 segments start more than 10^{-3} rad off the given configuration, 96 of them for that reason alone. This is PyRoki's start constraint not converging at the solver's default iteration limit.
  - 6 fail the check after slow-down.
- **As returned:** R1 passes on 0/480 segments. The knot trajectories exceed the jerk limit near the segment ends.

Inverse kinematics. Each goal pose of the held-out set is solved by PyRoki's IK variants and by cuRobo v1's IK solver. Success requires all of the following:

- the pose within 1 mm and 1°;
- within joint bounds;
- collision-free under the shared check.

Table 11: IK on the held-out goal poses (GPU).

	Success	Median time
PyRoki, default initial configuration	86/300	14 ms
PyRoki, 64 random starts	59/300	525 ms
PyRoki, collision-aware, 64 random starts	285/300	909 ms
cuRobo v1 IKSolver, default	298/300	36 ms
cuRobo v1 IKSolver, high precision	300/300	57 ms

- **Collision terms:** PyRoki's basic IK reaches the poses but often collides, since it has no collision terms.
- **Margin sensitivity:** the collision-aware variant uses the 5 mm margin chosen for trajectory optimisation.
  - At margin zero, 13 of its 15 failures succeed, but 3 of 28 control poses then collide.
  - This is reported as a sensitivity; the pre-registered number stands.

Free-space trajectory optimisation. PyRoki optimises 50-knot trajectories on the held-out queries from two kinds of seed:

- **S1:** the straight joint-space line;
- **S2:** the seeds cuRobo v1 itself used, attempt by attempt.

At the seed-dump random seed, cuRobo v1 solves 168/300 of the same queries.

- **Why the categories differ:** PyRoki has no global planner, so local optimisation from a seed is expected to fail where a passage is blocked. It is not ranked against the global planners.

Table 12: PyRoki trajectory optimisation on the held-out queries (GPU, one run per query).

	R1 success	R2 success	Median T_{ready}, R2 successes
S1: straight joint-space line	105/300	127/300	20.3 s
S2: cuRobo v1's seeds, in v1's order	104/300	169/300	25.0 s

- **Margin choice:** the world-collision margin was chosen on separate development problems by the rule "most R1 successes, ties to larger margin" (R1 successes out of 21: 0 mm: 7, 5 mm: 15, 10 mm: 14, 20 mm: 5, 50 mm: 1), giving 5 mm.

7 Limitations

- **One board.** Every number is from one Jetson Orin NX 16 GB. On a desktop GPU the balance between VAMP and cuRobo may differ, and nothing here measures that.
- **No UR controller in the loop.** No v2.2 program has been run on a UR controller (URSim). That leaves two gaps:
  - there is no comparison with the controller's own blends, arcs and profiles;
  - no planner trajectory has been streamed through `servoj`.

  URSim is published only for amd64, and this board is aarch64. URSim would in any case be kinematic: it checks the controller's trajectory generator and limits, not dynamics or tracking.
- **v2.2 program-set skew.** The generator kept 100 of 817 candidate cells (12%). A program had to have a continuous collision-free IK path and stay between 0.35 m and 0.70 m of the robot. The kept set is skewed:
  - **Vias:** W has 52 three-via programs against 24, 13 and 11 with four, five and six vias.
  - **Arcs:** the median arc sweep is 139° (range 90–269°).
  - **Clearance:** only 25 (W), 14 (C) and 24 (T) programs have less than 5 cm minimum clearance.

  Results on harder programs may differ.
- **Shared jerk limit.** The UR5e jerk limit is cuRobo's configuration value, since Universal Robots publishes none. A different limit changes which trajectories pass.
- **Load on the CPU.** VAMP's pick-place planning ran with other jobs on the CPU, and the v1 GPU slot shared the CPU with another job. Those times are upper bounds.
- **Subsampled cuRoboV2 runs.** cuRoboV2 runs on subsets, and its success rates in the v1 benchmark are lower bounds (`max_attempts` 20).
- **Our own components.** The Cartesian pipeline, the blend geometry and the v2.2 metrics are ours. Blend-zone deviation has no pass mark, because controllers blend differently.
- **PyRoki results are provisional** and come from one deterministic run per input.

8 Reproducibility

- **Code and data:** everything is in the vamp-bench repository.
  - The v2 and v2.2 problem sets are generated from the frozen commit `6aea6eecf76a`, with seeds derived from it.

  - The pre-registrations and every amendment are in `docs/`:
    - v2: `V2_PREREGISTRATION.md` and `V2_PICKPLACE_PREREGISTRATION.md`;
    - v2.1: `BENCHMARK_V2.md`;
    - v2.2: `V2_2_MOTION_TYPES_PREREGISTRATION.md`.
  - Corrections are in `CHANGELOG.md`.
- **Result files:**
  - v1: `results/summary.json` (`bench/analyze.py`);
  - v2: `results/v2/summary.json`, `hypotheses.json`, `pickplace_summary.json`;
  - v2.1: `results/v2_1/summary.json` (`bench/v2_1_summary.py`) and `results/v2_1/movel_r2.json`;
  - v2.2: `results/v2_2/summary.json` (`bench/v2_2_analyze.py`).
- **This report:** every number is generated from those files by `report/make_numbers.py`.
  - `bench/check_consistency.py` checks that the report source contains no typed numbers and that the generated files match the result files.
  - It also checks that every number in the built PDF comes from them.
- **Build:**

```
.venv/bin/python report/make_numbers.py
cd report && latexmk -pdf vamp_bench_report.tex
.venv/bin/python bench/check_consistency.py
```

- **Planner builds:** the VAMP build, cuRobo v1's Jetson fix and the cuRoboV2 build are described in the README and `docs/CUROBO_JETSON_FIX.md`.